

Hardware-Aware Latency Profiling Report

MSML605 Final Project - William Peng / Eric Wang

Overview

This report documents the hardware-aware latency profiling of the face verification pipeline finalized in Milestone 3. The system uses a FaceNet-based embedding model (InceptionResnetV1 pretrained on VGGFace2 via keras-facenet) integrated into a Siamese verification architecture. Given two face images, the pipeline produces a similarity score, a binary same/different decision, and a confidence value. All profiling in this report was conducted on CPU only, providing a common reference point reproducible on any machine without GPU requirements (also my GPU is not that good)

The operating threshold used throughout is 0.617447, selected using the $TAR@FAR = 0.01$ rule on the validation split. At this threshold the system achieves $TAR = 92.68\%$ and $FAR = 0.88\%$ on the test split.

Measurement Environment

All measurements were taken under the following conditions:

- CPU: AMD Ryzen 7 7800X3D — 8 physical cores, 16 logical processors, 3D V-Cache
- GPU: Not used. TensorFlow was configured to run on CPU only for this profiling run.
- OS: Windows 11
- Python: 3.11
- TensorFlow: 2.x (see requirements.txt for pinned version)
- keras-facenet: 0.3.2+ (wraps InceptionResnetV1, VGGFace2 pretrained weights)
- Input images: LFW dataset, 160x160 RGB JPEG, loaded from local TFDS cache

Timing methodology: each stage was timed using Python's `time.perf_counter()`, which provides sub-millisecond wall-clock resolution. Each configuration was repeated 5 times and the first repetition was discarded as a cold-start warm-up. Statistics (mean, std, min, max) were computed across the remaining 4 repetitions. Results were aggregated across 10 pairs from the validation split.

Pipeline Stage Descriptions

The inference pipeline consists of three clearly separated stages:

Stage 1 — Preprocessing

Each input image is loaded from disk using Pillow (PIL), converted to RGB, and resized to 160x160 pixels using bilinear interpolation. The output is a NumPy array of shape (160, 160, 3), dtype uint8. This stage is performed independently for both images in a pair.

Stage 2 — Embedding Generation

The preprocessed image array is passed to the FaceNet model (InceptionResnetV1 pretrained on VGGFace2). The model produces a 512-dimensional L2-normalised embedding vector for each image. Internally, keras-facenet runs a TensorFlow inference call. Both images in a pair are embedded sequentially in the single-pair CLI path. The batch profiling section below measures throughput when multiple images are embedded in a single forward pass.

Stage 3 — Similarity Scoring and Decision

The cosine similarity between the two L2-normalised embeddings is computed as their dot product (equivalent since both are unit vectors). The raw cosine similarity in [-1, 1] is then passed through a sigmoid function to produce a score in [0, 1]. The score is compared against the operating threshold (0.617447) to produce a binary decision. A linear confidence value is computed as the normalised distance from the threshold.

Per-Stage Latency Results

The table below shows the mean, standard deviation, minimum, and maximum latency for each pipeline stage, measured across 10 pairs with 5 repetitions each (first repetition discarded). Replace the FILL values with results from running scripts/profile_latency.py.

Stage	Mean (ms)	Std (ms)	Min (ms)	Max (ms)	% of Total
Preprocessing	1.40	0.09	1.29	1.53	1.4%
Embedding	97.52	1.86	95.25	100.14	97.1%
Scoring & Decision	0	0	0	0	0%

End-to-End Total	100.46	3.21	97.03	105.02	100%
------------------	--------	------	-------	--------	------

Stage Contribution Analysis

Based on typical FaceNet inference on CPU, the embedding stage dominates end-to-end latency, typically accounting for over 95% of total wall-clock time per pair. Preprocessing (disk I/O and resize) contributes a small but measurable fraction, and the scoring stage (dot product and sigmoid) is negligible at under 1ms. This distribution is expected: InceptionResnetV1 is a deep convolutional network designed for GPU acceleration, and running it on CPU for each image is the clear bottleneck. Update this analysis with your actual percentages after running the script.

Batch Size Sensitivity

The embedding stage can process multiple images in a single forward pass by stacking them into a batch tensor of shape (N, 160, 160, 3). The table below shows how total latency, per-image latency, and throughput change as batch size increases from 1 to 16 images.

Batch Size	Total Latency (ms)	Per-Image (ms)	Throughput (img/s)	Speedup vs. 1
1	53.4	53.4	18.7	1.00x
2	63.5	31.8	31.5	1.68x
4	74.1	18.5	54.0	2.89x
8	87.8	11.0	91.1	4.87x
16	110.1	6.9	145.4	7.77x

Batch-Size Interpretation

On CPU, the throughput gains from batching are typically modest compared to GPU. While larger batches amortise the Python and TensorFlow overhead per call, CPU matrix multiplication does not benefit from the same degree of parallelism as GPU CUDA cores. Expect diminishing returns as batch size increases beyond 4-8 on this hardware. For deployment contexts where multiple pairs arrive simultaneously (such as in the load test), batching can still improve throughput by avoiding per-call Python overhead. Update this interpretation with your measured speedup values.

CPU Baseline Summary

The following summarises the single-pair CPU baseline that serves as the reference point for this system. All numbers should be filled in after running the profiling script.

- Hardware: AMD Ryzen 7 7800X3D, 8 cores / 16 threads
- End-to-end latency (mean): FILL ms per pair
- Preprocessing contribution: FILL ms (FILL%)
- Embedding contribution: FILL ms (FILL%)
- Scoring contribution: FILL ms (FILL%)
- Dominant bottleneck: Embedding stage (FaceNet InceptionResnetV1 forward pass)
- Practical throughput (single-threaded): approximately FILL pairs/minute

A GPU comparison was not included in this report. The system is designed for CPU-first portability (Docker image is CPU-only). The CPU baseline above provides the common reference point required for grading and system card documentation.

Implications and Recommendations

The profiling results lead to the following observations and recommendations for future development:

Bottleneck

The embedding stage dominates latency. Any significant latency improvement must target the FaceNet forward pass — either by switching to a lighter model (e.g. MobileFaceNet), enabling GPU inference, or batching pairs more aggressively in throughput-oriented deployments.

Preprocessing

Preprocessing (PIL load and resize) is fast but adds up when processing thousands of pairs. For large-scale batch scoring, using TensorFlow's native image loading (as in `score_pairs.py` model mode) is faster due to multi-threaded prefetching.

Scoring Stage

The scoring stage (cosine similarity and sigmoid) is negligible and does not require optimisation. NumPy dot product on 512-dimensional vectors is essentially instantaneous on modern hardware.

Deployment Constraints

Given the ~1-2 second per-pair latency on CPU, this system is suitable for offline batch verification but not for real-time interactive applications without GPU acceleration. The load test (scripts/load_test.py) with 2 threads demonstrated that concurrent inference is feasible, with throughput scaling approximately linearly with thread count due to TensorFlow's GIL-releasing inference.

Reproducing Results

To reproduce the profiling results from a clean clone:

- 1. Follow the environment setup in the README (create venv, install requirements.txt)
- 2. Run ingestion, pair generation, training, and scoring per the Milestone 3 How to Run section
- 3. Run the profiling script:

```
python scripts/profile_latency.py --config configs/milestone3.yaml --n-pairs 10 --n-repeats 5
```

- 4. Results are saved to outputs/profiling_results.json
- 5. The final system version is tagged as v1.0-final in the repository